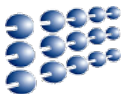




Facultatea
de Automatică
și Calculatoare
Universitatea Tehnică
din Cluj-Napoca



Fundamental Programming Techniques

Assignment 2 - Support Presentation (Part 1)

Computer Science Department, TUCN

Spring 2026



Problem and solution

PROBLEM: Improper queue management leads to high waiting times for clients and inefficient usage of resources.

SOLUTION: Queue management system implementing efficient queue allocation mechanisms.

How to design and implement the solution?

1. Clearly state the main objective and the sub-objectives required to reach it.
2. Analyze the problem and define the functional and non-functional requirements.
3. Design the solution.
4. Implement the solution.
5. Test the solution.





Objectives

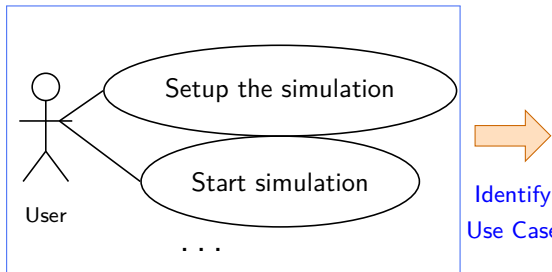
Main objective

- Design and implement an application aiming to analyze queuing-based systems by:
 - 1 simulating series of N clients arriving for service, entering Q queues, waiting, being served and finally leaving the queues
 - 2 computing the average waiting time, average service time and peak hour

Sub-objectives

- Analyze the problem and identify requirements
- Design the simulation application
- Implement the simulation application
- Test the simulation application

Analysis - Use Case Definition



Fragment of the Use Case Diagram

Use Case: setup simulation

Primary Actor: user

Main Success Scenario:

- The user inserts the values for the: number of clients, number of queues, simulation interval, minimum and maximum arrival time, and minimum and maximum service time.
- The user clicks on the validate input data button.
- The application validates the data and displays a message informing the user to start the simulation

Alternative Sequence: Invalid values for setup parameters

- The user inserts invalid values for the application; setup parameters
- The application displays an error message and requests the user to insert valid values
- The scenario returns to step 1

Analysis - Requirements Definition

Use Case: setup simulation

Primary Actor: user

Main Success Scenario:

- The user inserts the values for the: number of clients, number of queues, simulation interval, minimum and maximum arrival time, and minimum and maximum service time.
- The user clicks on the validate input data button.
- The application validates the data and displays a message informing the user to start the simulation

Non-Functional Requirements:

- The simulation application should be intuitive and easy to use by the user.
- ...What other non-functional requirements can you define?

Alternative Sequence:

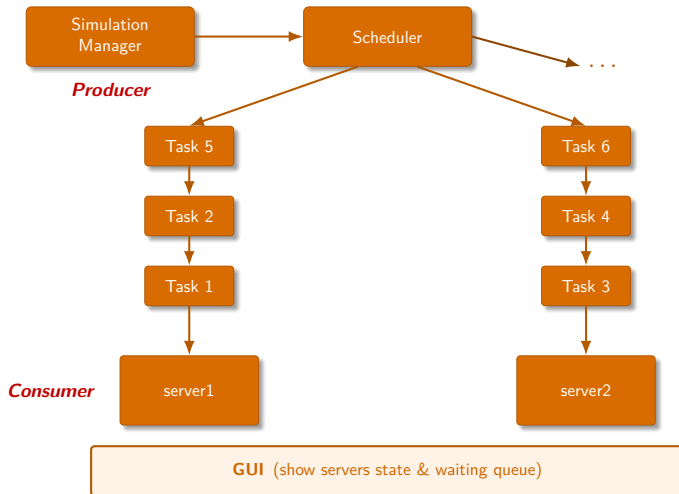
- Invalid values for setup parameters
- The user inserts invalid values for the application;s setup parameters
- The application displays an error message and requests the user to insert valid values
- The scenario returns to step 1

Functional Requirements:

- The simulation application should allow users to setup the simulation.
- The simulation application should allow users to start the simulation.
- The simulation application should display the real-time queues evolution
- ...What other functional requirements can you define?



Design - Conceptual Architecture



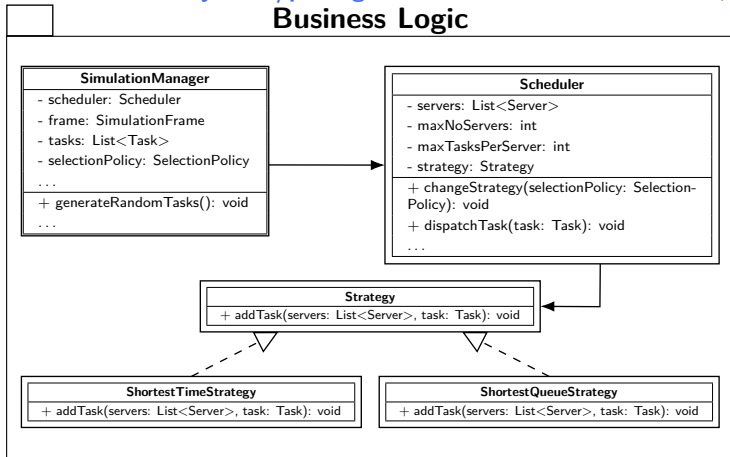


Detailed Design

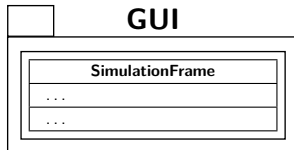
Division into sub-systems/packages

What other classes, attributes, methods are necessary?

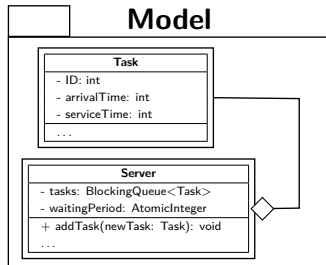
Business Logic



GUI

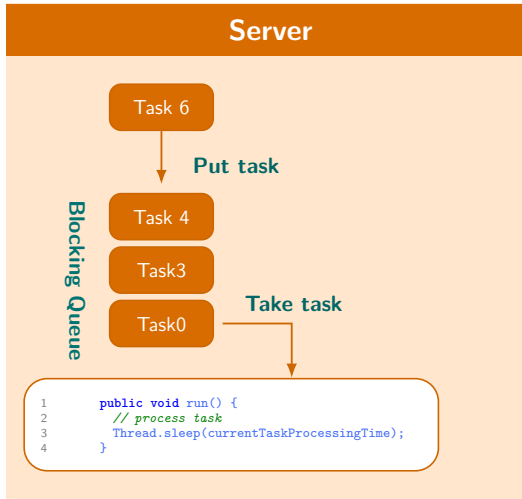


Model





Implementation - Server



```

1  public class Task {
2    private int arrivalTime;
3    private int serviceTime;
4    ...
5  }

6

7  public class Server implements Runnable {
8    private BlockingQueue<Task> tasks;
9    private AtomicInteger waitingPeriod;

10
11    public Server() {
12      // initialize queue and waiting period
13    }

14
15    public void addTask(Task newTask) {
16      // add task to the queue
17      // increment waitingPeriod
18    }

19
20    public void run() {
21      while (true) {
22        // take next task from queue
23        // stop the thread for a time equal with the task's
24          processing time
25        // decrement the waitingPeriod
26      }
27    }

28    public Task[] getTasks() {
29      ...
30    }
31  }
  
```




Implementation - Scheduler

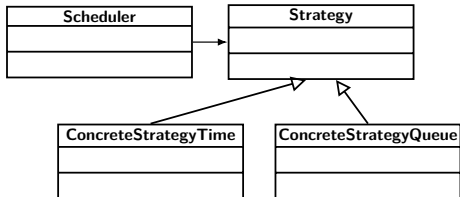
Sends tasks to Servers according to
the established strategy

```
1 public class Scheduler {
2     private List<Server> servers;
3     private int maxNoServers;
4     private int maxTasksPerServer;
5     private Strategy strategy;
6
7     public Scheduler(int maxNoServers, int maxTasksPerServer) {
8         // for maxNoServers
9         // - create server object
10        // - create thread with the object
11    }
12
13    public void changeStrategy(SelectionPolicy policy) {
14        // apply strategy pattern in order to instantiate the strategy
15        with the concrete strategy corresponding to policy
16        if (policy == SelectionPolicy.SHORTEST_QUEUE) {
17            strategy = new ConcreteStrategyQueue();
18        }
19        if (policy == SelectionPolicy.SHORTEST_TIME) {
20            strategy = new ConcreteStrategyTime();
21        }
22    }
23
24    public void dispatchTask(Task t) {
25        // call the strategy addTask method
26    }
27
28    public List<Server> getServers() {
29        return servers;
30    }
31 }
```

Implementation - Scheduler

• Scheduler - Strategy Pattern

- Choose the policy to distribute clients



```

1  public interface Strategy {
2      public void addTask(List<Server> servers, Task t);
3  }
4
5  public class ConcreteStrategyTime implements Strategy {
6      @Override
7      public void addTask(List<Server> servers, Task t) {
8          // TODO Auto-generated method stub
9      }
10 }
11
12 public enum SelectionPolicy {
13     SHORTEST_QUEUE,
14     SHORTEST_TIME
15 }
    
```



Implementation

Simulation Manager

- Generates randomly the tasks with:
 - ▶ Arrival time
 - ▶ Service time
- Contains simulation loop
 - ▶ Current time
 - ▶ Call scheduler to dispatch tasks
 - ▶ Update GUI

```

1 public class SimulationManager implements Runnable {
2     // following data read from UI
3     // maximum processing time, it is read from UI
4     public int timeLimit = 100;
5     public int maxProcessingTime = 10;
6     public int minProcessingTime = 2;
7     public int numberOfServers = 3;
8     public int numberOfClients = 100;
9     public SelectionPolicy selectionPolicy = SelectionPolicy.SHORTEST_TIME;
10
11     // entity responsible with queue management and client distribution
12     private Scheduler scheduler;
13     // frame for displaying simulation
14     private SimulationFrame frame;
15     // pool of tasks (client shopping in the store)
16     private List<Task> generatedTasks;
17
18     public SimulationManager() {
19         // initialize the scheduler
20         // => create and start numberOfServers threads
21         // => initialize selection strategy => createStrategy
22         // initialize frame to display simulation
23         // generate numberOfClients clients using generateNRandomTasks()
24         // store them to generatedTasks
25     }
26
27     private void generateNRandomTasks() {
28         // generate N random tasks;
29         // - random processing time
30         // minProcessingTime < processingTime < maxProcessingTime
31         // - random arrivalTime
32         // sort list with respect to arrivalTime
33     }
34 }

```

Implementation - Simulation Manager

```

1  @Override
2  public void run() {
3      int currentTime = 0;
4      while (currentTime < timeLimit) {
5          // iterate generatedTasks list and pick tasks that have the arrivalTime = currentTime
6          // - send task to queue by calling dispatchTask method from Scheduler
7          // - delete client from list
8          // update UI frame
9          currentTime++;
10         // wait an interval of 1 second
11     }
12 }
13
14 public static void main(String[] args) {
15     SimulationManager gen = new SimulationManager();
16     Thread t = new Thread(gen);
17     t.start();
18 }

```